MERU UNIVERSITY OF SCIENCE AND TECHNOLOGY

FIBONACCI SERIES IN DATA STRUCTURES

A Case Study of a Customer Loyalty Reward System

Course: Computer Security and Forensics

Prepared by Group 4:

1. Edward Mwaura — CT206/124754/25
2. Morris Mutuku — CT206/124744/25
3. Rayan Muchira — CT206/124740/25
4. Silvia Atieno — CT206/121980/25
5. Maxwell Lewis — CT206/124738/25

Lecturer: Dr. Dismus Kitaria

Department: Computer Science

Submission date: 16 October 2026

DECLARATION

We declare that this report was prepared by our group for the stated course. Any sources used have been listed in the references. The program and results should be checked by the group before submission.

ABSTRACT

This report looks at how the Fibonacci sequence can be used to demonstrate a simple customer loyalty reward system. It compares three ways of calculating Fibonacci numbers: basic recursion, an iterative method that stores values in an array and recursion with memoization. The program is written in C++ and lets a user enter the number of completed loyalty years before showing the points for each year. It also includes a small timing comparison using n = 30. The expected value is F(30) = 832,040. Actual running times are not included in the report because they should be measured on the computer used for the project. From the comparison, the iterative array method is the best fit for the example because the program needs to display every year's reward.

CHAPTER 1: INTRODUCTION

1.1 Background

The Fibonacci sequence is a number pattern where each new number is found by adding the two numbers before it. Using the convention in this report, it starts with 0 and 1, followed by 1, 2, 3, 5, 8, 13, and so on. It is a useful example in data structures and algorithms because different programs can calculate the same answer while doing very different amounts of work.

For this case study, we imagine a company that awards loyalty points based on how many complete years a customer has stayed with it. The points follow the Fibonacci sequence: Year 1 gets F(1), Year 2 gets F(2), and so on. This is only a classroom prototype. It does not manage real customer accounts or payments.

1.2 Problem statement

A program should calculate loyalty rewards correctly as the number of years increases. A simple recursive solution is easy to understand, but it repeats many calculations and becomes slow for larger values. The task is to implement and compare recursion, iteration with an array, and memoization, then select the method that best suits a program that must show the reward for every year.

1.3 Aim

To implement and compare different Fibonacci algorithms in C++ and use one of them in a small customer loyalty reward-system example.

1.4 Specific objectives

- Write a recursive function that calculates the nth Fibonacci number.
- Generate the Fibonacci sequence using a loop and an array-like vector.
- Use memoization to store previously calculated results.
- Create a C++ program that accepts loyalty years and prints each year's points.
- Compare the three methods for n = 30 without inventing execution-time results.
- Recommend an approach based on speed, memory use, and the need to display the whole sequence.

1.5 Research questions

- Why does basic recursion become slow as n increases?
- How do iteration and memoization reduce repeated calculations?
- What are the time and memory trade-offs of the three methods?
- Which method best suits a reward schedule that displays every year?

1.6 Scope and assumptions

- The project focuses on the algorithm and a console-based C++ program.
- The sequence is defined by F(0) = 0, F(1) = 1, and F(n) = F(n - 1) + F(n - 2).
- Loyalty Year 1 receives F(1) = 1 point.
- The program accepts 1 to 90 completed years. This range keeps the Fibonacci values and the total within the range of a typical signed 64-bit long long.
- The prototype does not include a database, login system, customer records or redemption features.
- Execution times vary by computer, compiler, build settings and background activity.

1.7 Importance of the study

This exercise is about how much work the program does, how much memory it uses and whether the data needs to be stored for later display.

CHAPTER 2: ALGORITHM DESIGN AND IMPLEMENTATION

2.1 Definition of the Fibonacci sequence

The sequence is defined as F(0) = 0, F(1) = 1, and F(n) = F(n - 1) + F(n - 2) for n ≥ 2. The reward schedule starts at F(1), because F(0) is the mathematical starting value rather than a loyalty year.

Index / Year	Fibonacci value	Meaning in this program
0	0	Starting value; not shown as a loyalty year
1	1	Year 1 reward
2	1	Year 2 reward
3	2	Year 3 reward
4	3	Year 4 reward
5	5	Year 5 reward
6	8	Year 6 reward
7	13	Year 7 reward

2.2 Method One: Basic recursion

Recursion is when a function calls itself to solve smaller versions of the same problem. For Fibonacci, the function returns n when n is 0 or 1. Otherwise, it adds the results of F(n - 1) and F(n - 2).

Pseudocode:

```
FIBONACCI-RECURSIVE(n)
  if n <= 1, return n
  otherwise return FIBONACCI-RECURSIVE(n - 1) + FIBONACCI-RECURSIVE(n - 2)
```

This method is short and follows the definition closely. Its problem is that the same terms are calculated repeatedly. For example, calls to F(5) and F(4) both end up calculating F(3) and smaller values.

Time complexity is exponential: O(2^n) is a common upper bound, while a tighter description is Θ(φ^n), where φ is approximately 1.618. The maximum recursion depth is O(n), so the call stack also uses O(n) space. This method is useful for learning, but not ideal for larger calculations.

2.3 Method Two: Iteration using an array

This method uses a loop instead of repeatedly calling a function. It stores the values in a vector, starting with F(0) = 0 and F(1) = 1. Each later value is calculated by adding the previous two values.

Pseudocode:

```
create fib[0..n]
fib[0] = 0; fib[1] = 1
for i from 2 to n:
  fib[i] = fib[i - 1] + fib[i - 2]
return fib
```

The loop calculates each value once. The running time is O(n), and the vector uses O(n) space. Since the program must display all yearly rewards, storing the sequence is useful: the reward for Year k can be accessed directly as fib[k].

2.4 Method Three: Memoization

Memoization is a top-down dynamic-programming technique. It still uses recursion, but saves each calculated result in a cache. If the same value is requested again, the function returns the saved result instead of calculating it from scratch.

Each Fibonacci term is calculated at most once, so the time complexity becomes O(n). The cache uses O(n) space, and the recursive call stack can also use O(n) space. Memoization is much faster than basic recursion for larger n, although it still has recursion overhead.

2.5 Tabulation

Tabulation is the bottom-up form of dynamic programming. It starts with known values and fills the table from smaller indices to larger ones. The iterative array method described above is also a tabulation approach. They are not separate methods in this implementation; they are two ways to describe the same bottom-up process.

If a program only needed F(n), rather than the whole sequence, a two-variable loop could calculate the answer in O(n) time and O(1) extra space. For this project, the array is more useful because all yearly rewards must be displayed.

2.6 Complexity comparison

Method	Time	Extra space	Main point
Basic recursion	O(2^n) upper bound	O(n) call stack	Simple, but repeats work
Iterative array	O(n)	O(n)	Stores the full sequence for display

Memoization	O(n)	O(n) cache + stack	Saves calculated values
Two-variable iteration	O(n)	O(1)	Useful when only the final value is needed

These complexity descriptions count operations under the assumption that arithmetic on a stored number takes constant time. Real integer types have fixed limits.

CHAPTER 3: CUSTOMER LOYALTY REWARD SYSTEM

3.1 How the program works

The user chooses the reward-schedule option and enters the number of completed loyalty years. The program checks that the input is a whole number between 1 and 90. It then generates the sequence, displays the reward for each year and adds the displayed rewards to produce a total.

3.2 Input, processing and output

Part	Description
Input	Completed loyalty years, from 1 to 90
Processing	Generate F(0) through F(years) with an iterative vector
Storage	Vector index y stores F(y)
Output	Each year's points and the total points across the displayed years

3.3 Example output

For an input of 5 years, the expected schedule is:

Year	Reward points
1	1
2	1
3	2
4	3
5	5
Total	12

The total is 1 + 1 + 2 + 3 + 5 = 12 points. This total is included to make the example more useful; the main requirement is to show the reward for each year.

3.4 Why use a vector?

A vector is a good fit because the number of years is known after input validation, and the program creates values in order. It also makes it easy to access and print a specific year's reward. In a real customer system, a database would be needed for customer details, saved reward history and an audit trail.

3.5 Input validation and number limits

The program rejects non-numeric input and values outside the permitted range. It uses long long rather than int because Fibonacci numbers grow quickly. The chosen limit of 90 keeps the values and total within the range of a signed 64-bit integer on typical C++ implementations. If the project needed larger values, it would need checked arithmetic or a big-integer library. The business should also decide whether this fast-growing reward rule makes financial sense.

3.6 Business considerations

- Fibonacci rewards rise quickly, so the company should check whether the reward policy is affordable.
- The value of a point, expiry rules, redemption conditions and maximum annual award are business decisions not defined by this prototype.
- A real system should protect customer information and keep records of changes to rewards.
- For many customers, the Fibonacci calculation is unlikely to be the main challenge; secure storage, correct business rules and reliable records would matter too.

CHAPTER 4: PERFORMANCE ANALYSIS

4.1 Purpose of the test

The program includes a small comparison of the three algorithms using n = 30. All methods should return F(30) = 832,040. This gives a known value for checking correctness while showing that algorithms returning the same answer can take different amounts of time.

4.2 Test environment

After running the C++ program, fill in the details below.

Item	Value to record
Computer / CPU	`Intel(R) Core(TM) i7-4600U CPU @ 2.10GHz`
Operating system	CachyOS Linux
C++ compiler and version	`g++ (GCC) 16.2.1 20260810`
Build command / settings	`g++ -std=c++17 -O2 FibonacciRewardSystem.cpp -o fibonacci_rewards`
Input n	30
Warm-up	3 calls per method
Measured repetitions	5 calls per method
Date and notes	14 October 2026

4.3 Procedure

1. Save the source file as FibonacciRewardSystem.cpp.
2. Compile it with a C++ compiler, for example: g++ -std=c++17 -O2 FibonacciRewardSystem.cpp -o fibonacci_rewards
3. Run it with: ./fibonacci_rewards (on Windows, run fibonacci_rewards.exe).
4. Choose option 2 to compare the algorithms at n = 30.
5. Check that the expected result is 832040 and record the displayed times.
6. Repeat the test if possible and describe any variation rather than choosing only the fastest result.

4.4 Results table

The Fibonacci result is known, but execution times depend on the test environment. Enter the measured values printed by the program.

Algorithm	n	Expected result	Measured average time
Basic recursion	30	832040	7 ms
Iterative array	30	832040	0.02 ms
Memoization	30	832040	0.03 ms

4.5 Discussion

Basic recursion should take much longer because it recalculates the same terms many times. The iterative method calculates each term once in a loop. Memoization also avoids repeated calculations, but it checks a cache and uses recursive calls. The exact time difference between iteration and memoization depends on the compiler, computer,.

This is a small classroom test, not a professional benchmark. It uses three warm-up calls and averages five measured calls. A stronger experiment would test more input sizes and use more repetitions. The report should describe what was actually measured instead of presenting the timings as universal results.

4.6 Limitations

- Execution time depends on hardware, compiler settings and other programs running at the same time.
- Five measured repetitions may not be enough for very short operations.
- The test uses only n = 30, so it does not show the full performance trend for many input sizes.
- The program uses fixed-width integer values and is not intended for arbitrarily large Fibonacci numbers.

CHAPTER 5: CONCLUSION AND RECOMMENDATIONS

5.1 Conclusion

This project compares three ways of generating Fibonacci numbers. Basic recursion is easy to follow but does repeated work and has exponential time complexity. Iterative array generation takes O(n) time and stores every value, while memoization also takes O(n) time by caching results but still uses recursion.

For the customer loyalty example, iterative array generation is the most suitable method because the program needs to print the reward for every year. It is straightforward, avoids recursive call-stack overhead and keeps the sequence available for direct access. The expected result for n = 30 is 832,040. The group should add the actual measured timing results after running the program.

5.2 Recommendations

- Use the iterative vector method for the reward schedule.
- Keep basic recursion for learning and comparison rather than for large inputs.
- Use memoization when a top-down recursive solution is useful.
- Test normal and boundary inputs, including 1, 2, 0, negative numbers, text and 90.
- Record the compiler, computer, build settings and actual timing results before submitting.
- Before using this idea in a real business, define reward caps, expiry, redemption and audit rules.

5.3 Possible future improvements

- Add automated tests for known Fibonacci values and invalid inputs.
- Benchmark several input sizes with more repetitions.
- Separate the calculation, input handling and display code into separate functions or classes.
- Use a database if customer information and reward history need to be saved.
- Use a big-integer type if exact values larger than the built-in integer range are required.

COMPLETE C++ SOURCE CODE

We have sent it as a separate file. The main features are basic recursion, iterative vector generation, memoization, input validation, a reward schedule and a simple timing comparison.

REFERENCES

- Cormen, T. H., Leiserson.; Introduction to algorithms.
- cppreference.com. (n.d.)